Smart Inventory Demand Forecasting Decision Support System

1st Kavyashri NP

Dept. of Computer Science and Engineering
CMR Institute of Technology
Bengaluru, India
kavyashri.np@cmrit.ac.in

2nd Pavan Raj BM

Dept. of Computer Science and Engineering
CMR Institute of Technology
Bengaluru, India
pabm23cs@cmrit.ac.in

3rd Pavan BV

Dept. of Computer Science and Engineering
CMR Institute of Technology
Bengaluru, India
pabv23cs@cmrit.ac.in

4th Ranjith Bhandari

Dept. of Computer Science and Engineering
CMR Institute of Technology
Bengaluru, India
rabh23cs@cmrit.ac.in

***Abstract*—Inventory planning requires a balance between product availability and the financial exposure associated with excess stock. Forecasting demand alone is insufficient when operational users must translate predictions into replenishment decisions, assess uncertainty, compare scenarios, and retain accountability for procurement actions. This paper presents a modular web-based Smart Inventory Demand Forecasting Decision Support System that connects supervised demand forecasting with inventory-risk calculations, explainability, scenario analysis, and a human-in-the-loop approval workflow.**

The implemented prototype uses a 48-row chronological dataset covering four products over twelve months and evaluates Linear Regression, Ridge Regression, Decision Tree, and Random Forest models using a 75:25 time-ordered split. In the reproducible baseline experiment, Ridge Regression obtained the lowest MAE of 13.39 and MAPE of 1.70%, while Linear Regression produced the highest R^2 of 0.9935. Ridge was therefore selected under the predefined MAE-first ranking criterion. The system further estimates forecast uncertainty, computes stockout and overstock indicators, derives safety stock and reorder points, supports what-if changes to demand-related inputs, and records managerial approval decisions. Because only 12 observations form the holdout set, the results are treated as preliminary evidence of prototype feasibility rather than evidence of general commercial forecasting performance. The contribution is consequently positioned as a practical forecast-to-decision integration rather than a new forecasting algorithm.

***Index Terms*—Demand forecasting, inventory management, decision support system, machine learning, Ridge regression, supply chain analytics, explainable AI, inventory optimization.**

I. Introduction

Inventory decisions are made under uncertainty: demand can change while inventory is already committed, supplier lead times can constrain replenishment, and price or sales conditions can alter the relationship between observed transactions and future demand. An under-estimated requirement can result in stockouts, delayed fulfilment, and lost revenue, whereas an over-estimated requirement can increase holding costs and tie up working capital. Recent research has therefore moved beyond isolated demand prediction toward approaches that connect predictive information with inventory decisions and supply-chain performance.

Machine-learning methods provide a flexible way of representing relationships among historical sales, inventory position, price, time, and supplier information. Recent demand-forecasting studies have investigated feature engineering, ensemble learning, deep learning, and hybrid architectures for heterogeneous product behaviour. At the inventory-control level, the literature increasingly considers learning methods as either a separate forecasting stage followed by optimization or as part of an integrated decision policy. These developments indicate that forecasting quality is important, but operational usefulness also depends on how predictions are interpreted and converted into actions.

Explainability is particularly relevant when a forecast contributes to a purchasing recommendation. Supply-chain decision makers may need to understand which variables influenced a prediction and how a change in demand or lead time would affect risk. Research on explainable artificial intelligence (XAI) in supply-chain decision support emphasizes the role of transparent predictive evidence in managerial decision making. At the same time, recent integrated approaches based on deep learning and reinforcement learning demonstrate the potential of more complex forecasting-to-control pipelines.

The present work takes a deliberately lightweight systems approach. Instead of introducing another forecasting architecture, it integrates four supervised regression models with a web application, a deterministic inventory decision layer, empirical uncertainty estimates, feature-sensitivity analysis, scenario simulation, and human approval. The resulting workflow is intended to support managerial judgement rather than replace it. This design also makes the assumptions of the inventory calculations visible to users.

The main contributions are: (1) a chronological multi-model demand-forecasting pipeline; (2) integration of forecast outputs with safety-stock, reorder-point, and order-quantity calculations; (3) empirical uncertainty and baseline-perturbation feature contribution estimates; (4) scenario-based sensitivity analysis; (5) human-in-the-loop recommendation approval with audit logging; and (6) a decoupled web architecture linking Node.js application services with Python machine-learning inference. The work is evaluated as a prototype and does not claim state-of-the-art forecasting performance or production-scale generalization.

II. Literature Review

A. Machine Learning for Demand Forecasting

Demand forecasting research has expanded from classical statistical models toward machine learning and deep learning. Mejía and Aguilar proposed a demand-forecasting system that first characterizes product time series, clusters products according to demand behaviour, and then evaluates forecasting methods and ensembles for the resulting groups. Their work illustrates the importance of product heterogeneity and feature engineering. Zhang *et al.* investigated hybrid attention-based deep-learning models for time-series product demand, reflecting the continued development of sequence-oriented forecasting methods. Riachy *et al.* addressed demand forecasting under large data gaps, showing that data availability and feature engineering remain important methodological concerns.

These studies motivate comparative model evaluation rather than assuming that one algorithm is universally suitable. The present prototype therefore evaluates four conventional supervised regressors using the same chronological holdout. The objective is not to outperform deep-learning systems, but to determine which lightweight model is most appropriate for the available prototype data and then connect that model to inventory decision logic.

B. Inventory-Aware Forecasting

The separation between forecasting and inventory control has increasingly been challenged. van der Haar *et al.* investigated supervised learning for integrated forecasting and inventory control and demonstrated that feature information can be incorporated directly into learning-based inventory decisions. Sukolkit *et al.* studied demand forecasting for steel inventory management and compared several forecasting architectures before integrating the forecasting outcome with inventory strategies. Abdi *et al.* considered demand forecasting within multi-objective, multi-product, and multi-period supply-chain optimization. Sarıcıoğlu *et al.* examined one-step and multi-step forecasting in relation to the bullwhip effect and supply-chain performance.

Bergsma *et al.* systematically reviewed machine-learning approaches in inventory-control optimization and distinguished approaches that use ML for separate estimation, integrated optimization, and dynamic policy learning. This classification places the current prototype closest to the separate-estimation/inventory-decision integration category: demand is forecast first, and the resulting estimate is passed to deterministic inventory logic. This is intentionally simpler than end-to-end inventory-policy learning.

C. Explainable AI and Decision Support

Explainability becomes important when predictions influence business decisions. Jauhar *et al.* examined explainable AI for resilient perishable-product supply chains using customer characteristics and transparent machine-learning techniques. Olana *et al.* reviewed the role of XAI capabilities in supply-chain decision-support systems and emphasized the importance of explanations in decision contexts. The present system does not implement formal Shapley-value attribution; instead, it uses a baseline-perturbation mechanism to estimate feature sensitivity. This distinction is retained explicitly because approximate feature contributions should not be interpreted as formal SHAP guarantees.

D. Advanced Integrated Approaches

Recent research also explores more complex architectures. Bo and Xu combine Transformer-based demand forecasting with particle-swarm optimization for supply-chain resilience and inventory-parameter optimization. Nguyen *et al.* investigate advanced time-series mixer models for forecasting and inventory under uncertain demand. Yang *et al.* combine multi-agent deep reinforcement learning with demand forecasting and inventory optimization in sensor-enabled retail supply chains. Mukherjee *et al.* examine machine-learning-driven demand forecasting for sustainable inventory control under uncertainty. Khan considers AI-enabled predictive forecasting in pharmaceutical inventory optimization.

E. Research Gap

The literature indicates several recurring challenges. First, many forecasting studies optimize prediction accuracy without presenting an operational workflow for converting the prediction into an auditable replenishment action. Second, integrated optimization and reinforcement-learning approaches can be computationally or operationally more complex than is necessary for a small prototype dataset. Third, explainability and uncertainty are often treated as analytical outputs rather than as inputs to a human review process. Finally, a practical gap remains between an offline forecasting experiment and a deployable application in which forecasting, inventory risk, scenario analysis, approval, and auditability are connected.

The proposed system addresses this integration gap without claiming that the individual components are novel. Its contribution is the engineering of a coherent pipeline in which a forecast becomes one piece of evidence in a traceable decision-support process.

TABLE I
SYNTHESIS OF RELEVANT LITERATURE

Ref.	Focus	Methodological emphasis	Relevance to this work
[1]	Forecasting and inventory control	Supervised learning with inventory-aware objectives	Motivates forecast-to-inventory integration
[2]	Supply-chain resilience	Transformer forecasting and PSO	Context for advanced predictive optimization
[3]	Product demand heterogeneity	Time-series features, clustering and ensembles	Supports comparative product-aware forecasting
[4]	Inventory demand forecasting	Comparative time-series models and inventory strategy	Connects forecast accuracy to inventory use
[5]	ML inventory optimization	Systematic review and taxonomy	Positions the prototype within ML-inventory approaches
[6],[7]	XAI and DSS	Explainability for supply-chain decisions	Motivates transparent evidence and human review
[8],[12]–[15]	Integrated optimization	Deep learning, time-series mixers, RL and applied AI	Establishes advanced alternatives and future directions

III. Problem Statement and Objectives

A. Problem Statement

Retail and warehouse operators need to decide when and how much inventory to replenish while demand, prices, inventory levels, and supplier lead times change. A forecast displayed without risk interpretation may not be sufficient for a procurement user. Conversely, a fully automated purchasing policy can act without contextual information that is not represented in historical data. The technical problem addressed here is therefore the design of a decision-support pipeline that transforms historical inventory observations into demand estimates and then into interpretable replenishment evidence while preserving human authority over the final action.

B. Objectives

The system objectives are to: (1) ingest and validate inventory-demand data; (2) evaluate multiple supervised regression models using chronological data separation; (3) estimate demand from sales, inventory, price, month, and supplier lead time; (4) serialize and invoke machine-learning inference from a Node.js application; (5) expose forecast metrics and uncertainty; (6) compute safety stock, reorder point, coverage and order quantity; (7) provide scenario simulation; (8) provide feature-sensitivity explanations; and (9) record approval, rejection, or snooze decisions in an auditable store.

C. Contributions

The contribution is a system-level integration rather than a new learning algorithm. The prototype combines a conventional supervised forecasting pipeline with deterministic inventory reasoning and an operational web workflow. This design allows the forecasting model to be replaced or extended without redesigning the entire decision-support layer.

IV. System Architecture and Software Engineering

The application uses a decoupled architecture consisting of a browser-based frontend, Node.js/Express backend, JSON persistence layer, and Python machine-learning process. The backend invokes the Python program through Node.js child-process execution and exchanges structured JSON payloads. Authentication and role-based authorization protect forecasting and procurement operations. The development database is implemented as JSON files, avoiding a native database dependency during the prototype stage.

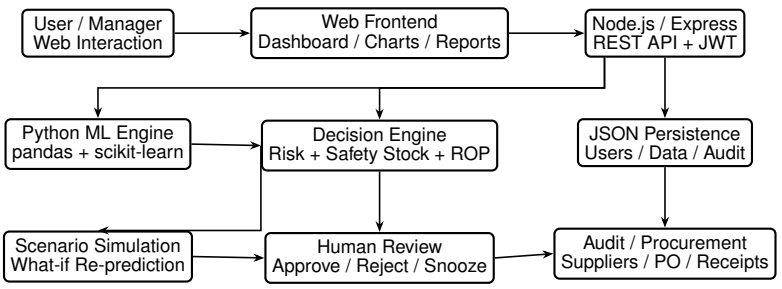


Fig. 1. Modular forecast-to-decision architecture of the implemented system.

A. Frontend Presentation Layer

The frontend is a browser-based single-page application implemented with HTML, CSS, and JavaScript. It provides dashboard views for demand intelligence, inventory intelligence, scenario analysis, model comparison, data quality, action review, audit information, and reporting. Chart.js is used for interactive visualizations. The interface is designed to present forecasts together with risk indicators and explanations rather than exposing only a raw predicted value.

B. Backend API Layer

The backend is implemented using Node.js and Express. REST endpoints expose authentication, forecasting, intelligence, scenario, audit, and procurement functions. JWT authentication is used for session authorization, while role checks restrict sensitive operations. The server invokes the Python forecasting process with structured JSON input and returns the resulting forecast, metrics, explanations, and risk information to the frontend.

C. Data Persistence and Schema Layer

The current implementation uses a lightweight JSON file store. The main persistence file contains users, scenarios, uploaded datasets, actions, suppliers, purchase requests, purchase orders, goods receipts, and inventory transactions. A separate audit store records decision history. This choice is appropriate for a local prototype but is not intended as evidence of production-scale database performance.

D. Machine Learning Inference Layer

The Python module uses pandas, NumPy, and scikit-learn. The default feature vector consists of sales, inventory, price, month, and lead time, with demand as the target. The system supports chronological splitting and multiple regression models. Model caching is used to avoid unnecessary repeated training when the dataset and selected features have not changed.

E. Decision Support Layer

Forecast output is passed to a deterministic risk engine. The application exposes predicted demand, stockout probability, safety stock, reorder point, inventory coverage, revenue-at-risk information, and recommendation categories. Scenario requests can modify demand-related inputs, price, inventory, or lead time and invoke a new prediction without permanently replacing the baseline dataset.

F. Human-in-the-Loop Workflow

Recommendations are not executed autonomously. An action can be approved, rejected, or snoozed. Rejection or delay can include a contextual reason, and the action record stores the recommendation and decision metadata. The procurement module additionally supports suppliers, purchase requests, purchase orders, receipt processing, and inventory transactions. This workflow makes the forecast an input to managerial action rather than an automatic procurement command.

V. Mathematical Formulation and Methodology

A. Dataset and Feature Engineering

The supplied baseline dataset contains 48 observations representing four products across twelve chronological months. The CSV contains product, sales, inventory, price, month, demand, lead-time, and category fields. For the reported multi-model comparison, the numerical forecasting features are sales, inventory, price, month, and lead time. Product and category fields are retained in the dataset but are not used as predictors in the baseline comparison. The target is unit demand.

The chronological order is preserved. With a 75:25 split, 36 observations are used for training and 12 for testing. This arrangement avoids random shuffling and prevents future observations from being used to train the reported holdout evaluation.

TABLE II
Feature Dictionary

Feature	Type	Operational meaning
Sales	Numeric	Historical sales input
Inventory	Numeric	On-hand inventory
Price	Numeric	Current selling price
Month	Numeric	Chronological month index
Lead time	Numeric	Supplier delivery duration
Demand	Numeric	Prediction target
Product	Categorical	Product identity in source data
Category	Categorical	Product category in source data

B. Data Quality Assessment

The quality module checks missing values, duplicate rows, invalid numeric values, negative values, required columns, and numerical outliers. Outliers are identified using the interquartile range (IQR):

$$IQR = Q_3 - Q_1 \tag{1}$$

A value below $Q_1 - 1.5IQR$ or above $Q_3 + 1.5IQR$ is flagged. The quality score is an engineering diagnostic and is not treated as a statistical measure of dataset validity.

C. Regression Models and Loss Functions

For Linear Regression, the parameter vector minimizes the residual sum of squares:

$$\min_w \|Xw - y\|_2^2 \tag{2}$$

Ridge Regression adds an L_2 penalty to mitigate overfitting by constraining the magnitude of the coefficients:

$$\min_w \|Xw - y\|_2^2 + \alpha\|w\|_2^2 \tag{3}$$

where the implementation uses $\alpha = 1.0$. Decision Tree Regression uses a maximum depth of five. Random Forest uses 100 trees for the reported comparison experiment, with a fixed random seed of 42.

The production-style forecast path also supports a 200-tree Random Forest for individual inference. This configuration difference is important: the 100-tree model is the one used for the reported four-model comparison, whereas the 200-tree configuration is an inference configuration. The two should not be treated as the same experiment.

D. Evaluation Metrics

Four standard regression metrics and forecast bias are reported. Mean Absolute Error is:

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i| \tag{4}$$

Root Mean Squared Error is:

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2} \tag{5}$$

For nonzero targets, MAPE is:

$$MAPE = \frac{100}{n}\sum_{i=1}^{n}\left|\frac{y_i - \hat{y}_i}{y_i}\right| \tag{6}$$

The coefficient of determination is:

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \overline{y})^2} \tag{7}$$

Forecast bias is defined by the implementation as mean$(\hat{y} - y)$, so negative values indicate under-forecasting on average.

E. Forecast Uncertainty Quantification

For Random Forest, the implementation calculates dispersion across individual tree predictions and combines it with training residual variability. For a nominal 90% level, the interval is represented as:

$$U_{90\%} = \hat{y} \pm 1.645\sigma \tag{8}$$

where σ represents the combined empirical uncertainty estimate. For non-tree models, residual standard deviation from chronological training residuals is used. These values are presented as empirical uncertainty estimates, not statistically calibrated prediction intervals, because coverage has not been validated on independent repeated samples.

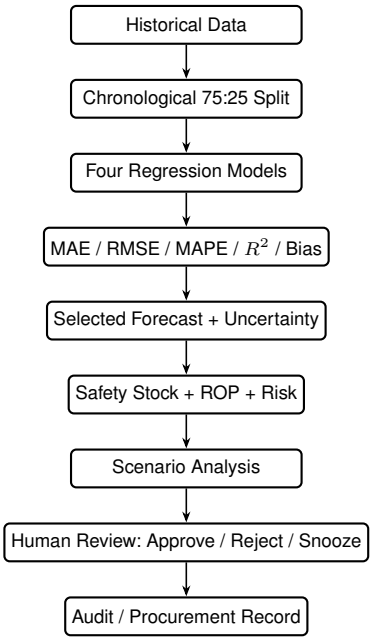


Fig. 2. Forecast-to-decision workflow used by the implemented system.

F. Feature Contribution Analysis Algorithm

The explanation mechanism is a baseline perturbation procedure. A baseline feature vector is formed from the mean feature values. The prediction at the baseline is compared with predictions generated after replacing one feature with the active observation while keeping other baseline values fixed. The difference is reported as a feature contribution. This method provides local sensitivity information but is not equivalent to exact Shapley-value decomposition.

To formalize this approach within the decision layer, Algorithm 1 defines the sensitivity attribution logic applied prior to rendering the user interface dashboard.

Algorithm 1 Baseline Perturbation Feature Sensitivity

Require: Model M, Current Vector X_c, Feature Set F
Calculate Baseline Vector $X_b \leftarrow \text{mean}(F_{train})$
Calculate Baseline Prediction $P_b \leftarrow M(X_b)$
Calculate Target Prediction $P_c \leftarrow M(X_c)$
for each feature $f \in F$ **do**
  $X_{temp} \leftarrow X_b$
  $X_{temp}[f] \leftarrow X_c[f]$
  $P_{temp} \leftarrow M(X_{temp})$
  Contribution $\Delta_f \leftarrow P_{temp} - P_b$
end for
return Array of Contributions Δ

G. Inventory Decision Logic

The forecast is converted from a monthly quantity to an approximate daily rate:

$$\overline{d} = \frac{\hat{D}}{30} \tag{9}$$

Under the simplifying assumption that supplier lead-time variability is negligible, safety stock is represented as:

$$SS = Z\sigma_d\sqrt{\overline{L}} \tag{10}$$

where Z is the service-level factor, σ_d is demand variability, and $\overline{L}$ is mean lead time. For a nominal 95% service level, $Z = 1.65$. The reorder point is:

$$ROP = \overline{dL} + SS \tag{11}$$

The prototype order-quantity rule is:

$$Q = \max(0, \hat{D} + SS - I) \tag{12}$$

where I is current inventory. This is a deterministic prototype policy, not a cost-optimal multi-echelon inventory-control formulation. Algorithm 2 defines how the deterministic risk engine transforms the machine learning inference into a procurement status string.

Algorithm 2 Deterministic Risk and Status Engine

Require: Forecast $\hat{D}$, Inventory I, Lead Time L
Calculate Safety Stock SS using Eq. 10
Calculate Reorder Point ROP using Eq. 11
Calculate Order Quantity Q using Eq. 12
if $I \leq SS$ **then**
  Status $\leftarrow$ REORDER_NOW
else if $I \leq ROP$ **then**
  Status $\leftarrow$ REORDER_SOON
else
  Status $\leftarrow$ MONITOR
end if
return Status, SS, ROP, Q

VI. Experimental Setup and Results

A. Hardware and Software Environment

The project is implemented as a full-stack web application. The backend uses Node.js 18+ and Express, while the machine-learning module uses Python with pandas, NumPy, and scikit-learn. The frontend is delivered as a browser application using HTML, CSS, JavaScript, and Chart.js. JSON files provide local persistence during development. The reported experiment is therefore a prototype evaluation rather than a distributed production benchmark.

B. Experimental Protocol

The baseline CSV contains 48 rows. The Python comparison routine sorts the records by month, separates the first 36 observations from the final 12 observations, trains all four regressors on the training portion, and computes metrics only on the holdout portion. The experiment is deterministic with fixed model random seeds where applicable. No random train-test shuffling is used.

TABLE III
Model Configuration

Model	Configuration
Linear Regression	Standardized inputs; ordinary least squares
Ridge Regression	Standardized inputs; $\alpha = 1.0$
Decision Tree	Maximum depth = 5; squared-error regression
Random Forest	100 estimators, bootstrap; random seed = 42

C. Computational Complexity Analysis

In addition to predictive accuracy, understanding the computational complexity of the evaluated models is crucial for scaling the prototype to a production environment tracking thousands of SKUs. Table IV outlines the theoretical time complexity for training and inference, where n is the number of historical records, d is the number of input features, and k represents the number of estimators (trees) in the Random Forest ensemble. Ridge Regression ensures linear inference time, satisfying the low-latency requirement of the Node.js API layer.

TABLE IV
COMPUTATIONAL COMPLEXITY OF EVALUATED MODELS

Model	Training Complexity	Inference Complexity
Linear Regression	$O(nd^2 + d^3)$	$O(d)$
Ridge Regression	$O(nd^2 + d^3)$	$O(d)$
Decision Tree	$O(nd \log n)$	$O(\text{depth})$
Random Forest	$O(k \cdot nd \log n)$	$O(k \cdot \text{depth})$

D. Model Comparison

The reproducible baseline comparison is shown in Table V. Ridge Regression obtains the lowest MAE and MAPE, while Linear Regression obtains the lowest RMSE and slightly higher R^2. The selection procedure ranks models primarily by MAE, followed by RMSE, MAPE, R^2, and absolute bias. Under this predefined ordering, Ridge is selected.

TABLE V
REGRESSION PERFORMANCE ON THE 12-OBSERVATION HOLDOUT SET

Model	MAE	RMSE	MAPE(%)	R^2	Bias
Random Forest	48.93	66.34	5.67	0.8955	-16.03
Linear Regression	13.87	16.58	2.00	0.9935	-10.57
Ridge Regression	**13.39**	17.70	**1.70**	0.9926	-9.95
Decision Tree	37.08	47.76	4.52	0.9458	-5.42

E. Error Analysis and Model Selection

Ridge Regression improves on Linear Regression in MAE and MAPE, but not in RMSE or R^2. The difference is therefore criterion-dependent rather than evidence that Ridge dominates every measure. The model selection is intentionally based on MAE because MAE expresses the average forecasting error in the original demand units and is easier to relate to inventory planning.

The tree-based models perform substantially worse on the holdout set. With only 36 training observations, non-parametric models have limited evidence from which to learn stable partitions of the feature space. Ridge introduces regularization and therefore imposes stronger structural constraints, which can be advantageous when data are scarce and predictors are correlated. This interpretation is specific to the current experiment and should not be generalized to larger commercial datasets.

F. Interpretation Caveat

The reported R^2 values above 0.99 for the two linear models should be interpreted cautiously. They arise from a very small, structured dataset and a single 12-observation test window. The experiment demonstrates that the implemented data flow can train models, compare their errors, select a model, and pass predictions to the decision layer. It does not establish that the same error rates would be obtained on larger, noisier, seasonal, or highly volatile inventory datasets.

VII. DECISION SUPPORT FEATURES AND ANALYSIS

A. Demand Intelligence and Inventory Risk

The dashboard combines the forecast with inventory context. The decision layer derives safety stock, reorder point, inventory coverage, stockout probability, and revenue-at-risk information. This allows a user to interpret a forecast in relation to current stock rather than treating the forecast as an isolated number. Recommendation states can include REORDER_NOW, REORDER_SOON, HOLD, REDUCE_ORDER, and MONITOR, subject to the implemented risk rules.

B. Scenario Simulation Engine

The scenario engine accepts controlled changes to demand-related inputs, including sales or demand assumptions, price, inventory, and lead time. The modified feature vector is passed through the forecasting process and the resulting demand and risk indicators are compared with the baseline. Importantly, scenario changes are treated as hypothetical inputs and do not overwrite the underlying baseline dataset.

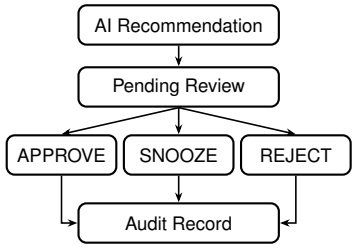


Fig. 3. Human-in-the-loop action and audit workflow.

C. Case Study A: Supply Chain Lead-Time Shock

To validate the scenario simulation capabilities, a theoretical supply shock was modeled where a critical supplier's lead time increases from 5 to 15 days. By adjusting the $\overline{L}$ parameter in the mathematical formulation, the system dynamically recalculates the baseline safety stock.

As shown in Table VI, the tripling of lead time drastically altered the risk profile. The UI recommendation immediately shifted from MONITOR to REORDER_NOW. This demonstrates the practical utility of integrating deterministic safety stock equations with predictive demand variables.

TABLE VI
SCENARIO A: IMPACT OF LEAD TIME SHOCK

Metric	Baseline ($L = 5$)	Scenario ($L = 15$)
Forecasted Demand ($\hat{D}$)	120 units	120 units
Daily Rate ($\overline{d}$)	4 units/day	4 units/day
Safety Stock (SS)	25 units	43 units
Reorder Point (ROP)	45 units	103 units
Current Inventory (I)	55 units	55 units
Status Recommendation	**MONITOR**	**REORDER_NOW**

D. Case Study B: Price Elasticity Adjustment

A second scenario involved simulating a 15% reduction in the current selling price to anticipate a holiday promotion. Using the baseline-perturbation explanation mechanism (Algorithm 1), the manager could isolate the direct impact of the price drop on expected unit demand. The deterministic inventory engine subsequently adjusted the forecasted depletion rate $\overline{d}$ and elevated the inventory coverage risk, preventing a potential stockout event before human approval was even required.

E. Human-in-the-Loop Approval

A recommendation is placed into an action workflow rather than immediately creating an autonomous procurement action. The user can approve, reject, or snooze the recommendation. A reason can be captured for rejection or delay. This mechanism is useful because managers may possess information about promotions, supplier relationships, upcoming events, or business constraints that is not represented in the historical feature set.

F. Auditability and Procurement

The backend records actions in persistent JSON structures and supports procurement entities including suppliers, purchase requests, purchase orders, goods receipts, and inventory transactions. Purchase orders are protected by approval and role checks. Status transitions are explicitly controlled, allowing the system to preserve a trace of procurement progress. These capabilities extend the forecasting prototype toward a more complete operational workflow while remaining dependent on the limitations of local JSON persistence.

G. AI Copilot

The implemented intelligence layer also exposes a natural-language Copilot interface for predefined inventory queries, such as identifying products at stockout risk or asking what should be reordered. Its responses are derived from application data and decision logic rather than being used as the forecasting model itself. This distinction prevents the conversational interface from becoming an unsupported source of numerical predictions.

VIII. Scalability and System Deployment

While the current architecture validates the end-to-end integration of machine learning and deterministic inventory rules, scaling this system for enterprise-grade deployment requires strategic architectural shifts.

A. Database Migration

The reliance on local JSON persistence is inherently restricted by file I/O locks and memory constraints. Future iterations will migrate the data layer to PostgreSQL. This transition will provide ACID compliance for concurrent procurement workflows, allowing multiple managers to approve purchase orders simultaneously without write collisions.

B. Asynchronous Task Processing

Invoking the Python ML engine as a synchronous Node.js child process introduces latency bottlenecks. To address this, the Python layer will be containerized via Docker and deployed as an independent microservice using Flask or FastAPI. Message brokers such as Redis or RabbitMQ will be utilized to handle asynchronous batch inference for thousands of SKUs, preventing UI freezing during heavy scenario simulations.

IX. Challenges and Limitations

The primary limitation is the small evaluation dataset. Only 48 observations are available, and only 12 observations form the final holdout set. Such a short horizon is insufficient for reliable assessment of multi-year seasonality, holiday effects, structural breaks, promotions, or broad market volatility. A single chronological holdout can also be sensitive to the particular final three months selected for testing.

A second limitation concerns the persistence architecture. JSON files are useful for local development and demonstration, but they do not provide the concurrency, transactions, indexing, backup, and access controls expected from a production multi-user database. A relational database should therefore be evaluated before operational deployment.

Third, the forecasting formulation does not explicitly model censored demand. If an item was unavailable, observed sales could understate true customer demand. The current pipeline also uses a limited set of explanatory variables and does not incorporate holidays, promotions, competitor prices, macroeconomic indicators, or external demand signals.

Fourth, uncertainty estimates are empirical rather than calibrated. The Random Forest implementation combines tree-level dispersion with training residual variability, while non-tree models use residual-based estimates. Coverage is not established experimentally. Similarly, the feature-attribution mechanism is a baseline perturbation approximation rather than exact SHAP.

Finally, the order-quantity rule is deterministic and intentionally lightweight. It does not optimize explicit holding costs, stockout penalties, supplier constraints, multi-echelon interactions, or service-level trade-offs. The system should therefore be understood as decision support rather than a complete inventory-control optimizer.

X. Future Research Directions

The first priority for future work is a larger multi-year SKU dataset with sufficient temporal diversity for repeated out-of-sample evaluation. Rolling-origin or walk-forward validation should replace reliance on a single holdout window. Such evaluation would provide several independent test periods and a stronger estimate of temporal generalization.

The software architecture can be extended from JSON persistence to PostgreSQL or another transactional database. Additional variables such as promotions, holidays, stockout indicators, supplier reliability, and external market signals can be evaluated when reliable data are available. Product-level segmentation and richer time-series features could also be introduced to address heterogeneous demand behaviour.

Methodologically, future experiments can compare the current regression baseline with LSTM, GRU, attention-based, time-series mixer, or reinforcement-learning approaches. Such comparisons should be conducted only after sufficient data and a consistent temporal evaluation protocol are available. Prediction intervals should also be calibrated empirically, and formal explainability methods such as SHAP can be evaluated against the current perturbation approximation.

At the inventory-control level, future versions can move from deterministic replenishment rules toward cost-aware optimization incorporating holding cost, stockout cost, service-level constraints, supplier capacity, and multi-echelon interactions. These extensions would allow the system to progress from a transparent prototype toward a more rigorous inventory-control platform. Furthermore, the integration of Large Language Models (LLMs) to enhance the AI Copilot's contextual reasoning capabilities will be explored to provide unstructured data insights (e.g., supplier emails, news events) directly to the procurement manager.

XI. Conclusion

This paper presented a Smart Inventory Demand Forecasting Decision Support System that integrates supervised demand forecasting with inventory-risk calculations, scenario simulation, feature-sensitivity information, and a human-in-the-loop operational workflow. The implementation connects a browser-based frontend to a Node.js/Express backend and a Python scikit-learn forecasting process, while lightweight JSON persistence provides the prototype data and audit layer.

On the 48-row baseline dataset, four regression models were compared using a chronological 75:25 split. Ridge Regression achieved the lowest MAE of 13.39 and MAPE of 1.70%, and was selected under the predefined MAE-first ranking. Linear Regression obtained a slightly higher R^2 of 0.9935 and lower RMSE than Ridge, illustrating why model selection should be tied to an explicit criterion rather than to a single headline metric. The results are preliminary because only twelve observations form the holdout set.

The principal contribution is therefore the integration of forecasting into an auditable decision-support workflow. The system demonstrates how a demand estimate can be accompanied by uncertainty, feature sensitivity, inventory-risk indicators, scenario analysis, and managerial approval rather than being presented as an isolated prediction. Larger datasets, repeated temporal validation, calibrated uncertainty, richer demand variables, and cost-aware inventory optimization are required before production-scale or generalization claims can be justified.

References

[1] J. F. van der Haar, A. P. Wellens, R. N. Boute, and R. J. I. Basten, "Supervised learning for integrated forecasting and inventory control," *European Journal of Operational Research*, vol. 319, no. 2, pp. 573-586, 2024.

[2] L. Bo and J. Xu, "Enhancing supply chain efficiency resilience using predictive analytics and computational intelligence techniques," *IEEE Access*, vol. 12, pp. 183451-183465, 2024.

[3] S. Mejía and J. Aguilar, "A demand forecasting system of product categories defined by their time series using a hybrid approach of ensemble learning with feature engineering," *Computing*, vol. 106, pp. 3945-3965, 2024.

[4] N. Sukolkit, S. Arunyanart, and A. Apichottanakul, "An open innovative inventory management based demand forecasting approach for the steel industry," *Journal of Open Innovation: Technology, Market, and Complexity*, vol. 10, no. 4, Art. no. 100407, 2024.

[5] R. Bergsma, C. de Ruijt, and S. Bhulai, "A systematic review of machine learning approaches in inventory control optimization," *Operations Research Perspectives*, vol. 15, Art. no. 100367, 2025.

[6] S. K. Jauhar, S. Harinath, V. Krishnaswamy, and S. K. Paul, "Explainable artificial intelligence to improve the resilience of perishable product supply chains by leveraging customer characteristics," *Annals of Operations Research*, vol. 354, pp. 103-142, 2025.

[7] F. Olana, K. Spanaki, W. Ahmed, and G. Zhao, "Enabling explainable artificial intelligence capabilities in supply chain decision support making," *Production Planning & Control*, pp. 808-819, 2024.

[8] F. Abdi *et al.*, "A machine learning-based approach for multi-objective, multi-product, and multi-period supply chain optimization via demand forecasting," *IEEE Access*, vol. 13, pp. 147343-147368, 2025.

[9] X. Zhang *et al.*, "Enhancing time series product demand forecasting with hybrid attention-based deep learning models," *IEEE Access*, vol. 12, pp. 190079-190091, 2024.

[10] C. Riachy *et al.*, "Enhancing deep learning for demand forecasting to address large data gaps," *Expert Systems with Applications*, vol. 268, Art. no. 126200, 2025.

[11] A. Sarıcıoğlu, M. E. Genevois, and M. Cedolin, "Analyzing one-step and multi-step forecasting to mitigate the bullwhip effect and improve supply chain performance," *IEEE Access*, vol. 12, pp. 180161-180174, 2024.

[12] T. N. A. Nguyen *et al.*, "Optimizing supply chain operations using advanced time-series mixer models for demand forecasting and inventory under uncertain demand," *Expert Systems with Applications*, vol. 296, Art. no. 128955, 2026.
[13] A. K. Mukherjee *et al.*, "Machine learning-driven demand forecasting for multi-objective sustainable inventory control under uncertainty," *International Journal of Management Science and Engineering Management*, 2026.
[14] Y. Yang *et al.*, "Multi-agent deep reinforcement learning for integrated demand forecasting and inventory optimization in sensor-enabled retail supply chains," *Sensors*, vol. 25, no. 8, Art. no. 2428, 2025.
[15] R. Khan, "AI-enabled predictive forecasting for inventory optimization in pharmaceutical supply chains," *World Journal of Advanced Engineering Technology and Sciences*, vol. 17, no. 2, 2025.